

# MICA

## Commercial

---

The customer commitment specialist. Explains what customers ordered, what was promised, what has shipped and what service or warranty obligation remains.

**Made for:** product, engineering, operations and business teams.  
**Reading time:** about 12 minutes.  
**Truth standard:** describes the repository as implemented on 1 August 2026.

MICA

# What MICA is here to do

A role, not an all-powerful chatbot

## In one sentence

Explains what customers ordered, what was promised, what has shipped and what service or warranty obligation remains.

**2**

product areas grouped here

**2**

registered callable tools

**3**

allowed capability prefixes

**0**

agents it may delegate to

## Receives

Customer order and requested date · Credit and dispatch state · Ticket, SLA and entitlement state · Supply/plan/quality answers from peer agents

## Produces

Commitment view · Customer-risk evidence · Commercial follow-up work item · Clear statement of what is and is not yet promised

## Allowed capability families

sales.\*

customer.\*

agent.\*

## The key distinction

The product-area grouping below shows where MICA contributes across XELOR. The callable-tool table later shows the smaller, exact surface Agent OS can invoke today. Product ownership does not automatically create runtime authority.

# The business areas under MICA

Business ownership is broader than today's Agent OS tools

## Sales

Customers, GST-aware orders, credit checks, requested delivery dates and dispatch.

## Service / CSP

Customer-isolated tickets, SLA clocks, warranty, complaints, spares, knowledge and reviewable reply drafts.

### How to read this grouping

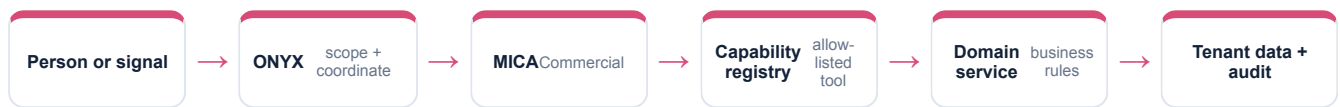
MICA is the named specialist when a mission needs facts or analysis from these areas. Each module still owns its own rules, records and write paths. The agent coordinates through registered services; it does not become the database owner.

### What remains true underneath

- Each module keeps its own business rules and permissions.
- Cross-module work passes through narrow ports or registered capabilities.
- Tenant isolation, audit and workflow apply regardless of which agent is speaking.
- A readable agent answer never replaces the source record.

# How MICA participates in a mission

The mission engine controls order, parallel work and recovery



## 1 · Scope

ONYX turns the goal into a versioned graph run with a fixed tenant, user, limits and expected outputs.

## 2 · Authorise

The capability registry checks that MICA is allowed to call the named tool; RBAC repeats the user permission check.

## 3 · Execute safely

The registered domain service performs the operation. The agent never receives a general SQL doorway.

## 4 · Preserve evidence

Inputs, outputs, node events and checkpoints are stored so the mission can be explained or recovered.

## Delegation

MICA is a specialist and does not delegate to other agents in the current registry.

# What MICA can call today

Every row is explicitly registered and permission-checked

| Capability key        | Mode    | Result             | User permission     | Approval? |
|-----------------------|---------|--------------------|---------------------|-----------|
| sales.orders.read     | Read    | Sales orders       | sales.order.read    | No        |
| agent.action.dispatch | Execute | Governed work item | agentos.run.operate | Yes       |

## What “dispatch” means

If listed, `agent.action.dispatch` appends a governed work item for a named specialist. It does not directly change a sales order, stock balance, production order, journal, payment or customer message.

### Read / analyse / simulate

Returns evidence or a reproducible view. No business record is changed.

### Draft

Creates reviewable proposed content. A draft is never labelled as an approved or completed business outcome.

### Execute

Effectful at the Agent OS level and therefore requires an approved ancestor node.

### Permission

The agent's allow-list and the signed-in user's RBAC must both allow the call.

# Who MICA works with

Specialists exchange evidence through ONYX, not hidden side conversations

**AXLE**

Passes dated customer demand into planning.

**SPAR + KILN**

Uses availability, production and quality evidence before calling a promise safe.

**RASP**

Shares order, credit, receivable and collection context.

## A complete controlled mission



### Why this structure matters

Each agent stays accountable for its own facts. ONYX combines them, HEXA checks control evidence, and a person controls the decision boundary. This prevents one fluent answer from quietly becoming authority over the whole company.

# Which customer promise is at risk?

A realistic presentation path from question to controlled outcome

1

MICA reads open sales orders and keeps the customer's requested date visible instead of inventing a new date.

2

ONYX joins that commitment with AXLE's exception, SPAR's material position and KILN's production/inspection state.

3

MICA can receive an approved follow-up work item, but the current Agent OS does not silently edit the sales order or message the customer.

## What the presenter should say

"MICA contributes verified domain evidence, with source references and limitations. It does not claim that a proposed or dispatched action is complete until the controlled system records and verifies the outcome."

## Useful questions to ask during the demo

- Which source records support MICA's answer?
- What permission was checked?
- Is this a fact, a simulation, a draft or a completed result?
- Would this step wait for human approval?
- What happens if a node times out or the service restarts?

# How MICA stays trustworthy

Controls are enforced in the runtime and modules, not left to prompt wording

## Control 1

Customer portal rows have both tenant and customer-account isolation

## Control 2

Sent replies are frozen and drafts cannot make promises

## Control 3

Dispatch depends on inventory and quality gates

## Control 4

Commercial actions still inherit user RBAC

### Identity boundary

Verified user + tenant

### Authority boundary

Agent allow-list + user RBAC

### Action boundary

Approval ancestry for side effects

### Data boundary

Domain service + tenant-fenced database

### Evidence boundary

Append-only run, event and audit records



# What is live—and what is not

This page prevents the demo from overclaiming

## Live in the repository

- Sales-order evidence
- GST and credit rules
- Dispatch controls
- SLA and warranty computation
- Customer risk contribution to missions

## Deliberate current limits

- Agent OS currently exposes sales-order read, not every Sales/CSP screen
- No automatic customer email or portal message
- Work items require a human owner to complete downstream work

## Runtime disclosure

Orchestration, ERP reads, approval gates and governed action dispatch are live. Language reasoning is deterministic; no external model API or connector is active.

## Primary implementation references

`apps/api/src/modules/sales/`

`apps/api/src/modules/csp/`

`apps/api/src/agent-os/capability-registry.service.ts`

This guide is a plain-language operating map, not an API contract. The code, migrations and automated tests remain the binding technical source.

# MICA in one page

Use this page when explaining the agent to a new team member

## Job

Explains what customers ordered, what was promised, what has shipped and what service or warranty obligation remains.

## Owns

Sales, Service / CSP

## Receives

Customer order and requested date · Credit and dispatch state · Ticket, SLA and entitlement state · Supply/plan/quality answers from peer agents

## Returns

Commitment view · Customer-risk evidence · Commercial follow-up work item · Clear statement of what is and is not yet promised

## Can call

sales.orders.read, agent.action.dispatch

## Cannot do

Agent OS currently exposes sales-order read, not every Sales/CSP screen · No automatic customer email or portal message · Work items require a human owner to complete downstream work

## Plain-language glossary

|                           |                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------|
| <b>Agent</b>              | A named specialist role with an allow-listed set of tools—not a free-roaming process.    |
| <b>Capability</b>         | One registered operation with an input, output, owner, mode and required permission.     |
| <b>Graph</b>              | A versioned mission recipe showing order, parallel steps, joins and approvals.           |
| <b>Checkpoint</b>         | A stored progress snapshot used for explanation and safe recovery.                       |
| <b>Governed work item</b> | An append-only assignment created after approval; it is not the final business mutation. |